UPPSALA
UNIVERSITET

Parallel and Distributed Programming

Assignment-1

Assignment-1(Parallel Global Sum using MPI)

Submitted by

Rajib Datta
rajib.datta.3822@student.uu.se

Department of Information Technology

April 23, 2026

Contents

1 Introduction **3**

2 Parallelization Approach **3**
2.1 Block Partitioning (Task 2.1) 3
2.2 Tree-Structured Global Sum (Task 2.2) 4

3 Performance Experiments (Task 2.2.1) **4**
3.1 Strong Scaling 4
3.2 Weak Scaling 5

4 Cost Analysis (Task 2.3) **5**

5 Speedup and Efficiency Analysis (Task 2.4) **7**
5.1 Observations 8

6 Scalability Analysis (Task 2.5) **8**

7 Discussion **9**

8 Conclusion **9**

9 References **10**

1 Introduction

Parallel computing allows computational problems to be solved faster by dividing work among multiple processors. One common operation in scientific computing is the computation of a **global sum** across distributed data. This operation is also known as **parallel reduction**.

In this assignment, a parallel program was implemented using the **Message Passing Interface (MPI)** to compute the sum of a large array. The array elements are distributed among processes, each process computes a local sum, and the results are combined using a **tree-structured reduction algorithm**.

The objectives of this assignment are:

- Implement block partitioning of an array among processes
- Implement a tree-based global sum algorithm using MPI
- Perform strong and weak scaling experiments
- Analyze speedup, efficiency, and scalability

All experiments were conducted on a Linux cluster using MPI and the C programming language.

2 Parallelization Approach

The parallel program consists of two main stages:

1. Data partitioning across processes
2. Parallel reduction to compute the global sum

2.1 Block Partitioning (Task 2.1)

Assume the array contains n elements and the computation uses p processing units. The array is divided into p blocks so that each process handles approximately n/p elements.

For process k, the first and last indices are computed as

$$first_k = \frac{k \cdot n}{p}$$

$$last_k = \frac{(k+1) \cdot n}{p} - 1$$

This ensures a balanced workload distribution across processes.

Each process generates random numbers and computes a local partial sum of its assigned block. This partitioning strategy is implemented in the program using:

```
long long int first = (rank * n) / size;
long long int last  = ((rank + 1) * n) / size - 1;
```

2.2 Tree-Structured Global Sum (Task 2.2)

After computing local sums, the processes combine the results using a tree-based reduction algorithm.

The algorithm works in $\log_2(p)$ communication steps. At each step, half of the processes send their partial sums to the other half.

For example, with $p = 8$:

- Step 1: 1→0, 3→2, 5→4, 7→6
- Step 2: 2→0, 6→4
- Step 3: 4→0

After the final step, process 0 holds the global sum.

The communication is implemented using the MPI functions:

- `MPI_Send`
- `MPI_Recv`

This method reduces communication complexity to $O(\log p)$.

3 Performance Experiments (Task 2.2.1)

To evaluate the performance of the parallel program, two types of scalability experiments were conducted.

3.1 Strong Scaling

Strong scaling measures how execution time decreases when the number of processors increases while keeping the problem size constant.

Experiment configuration:

- Problem size: $n = 2^{22}, 2^{23}, 2^{24}, 2^{25}, 2^{26}$
- Processes: $p = 1, 2, 4, 8$

Execution time was measured using the MPI timing function:

MPI_Wtime ( )

Efficiency was computed using

$$E = \frac{T_1}{pT_p}$$

where T_1 is the sequential runtime and T_p is the parallel runtime. Expected behavior:

- Speedup increases as processors increase.
- Efficiency decreases slightly due to communication overhead.

3.1.1 Execution Time Results for Strong Scaling

Table 1 shows example results from the strong scaling experiment.

Figure 1 shows the strong scaling efficiency and speedup of different problem sizes under 1,2,4,8 processes.

Table 1: Strong Scaling Execution Times

Processes (p)	Problem Size	Time (s)
1	2^{22}	0.026
2	2^{22}	0.013
4	2^{22}	0.006
8	2^{22}	0.004
1	2^{23}	0.052
2	2^{23}	0.026
4	2^{23}	0.013
8	2^{23}	0.006
1	2^{24}	0.104
2	2^{24}	0.052
4	2^{24}	0.026
8	2^{24}	0.013
1	2^{25}	0.207
2	2^{25}	0.104
4	2^{25}	0.052
8	2^{25}	0.033
1	2^{26}	0.414
2	2^{26}	0.207
4	2^{26}	0.104
8	2^{26}	0.052

3.2 Weak Scaling

Weak scaling measures performance when the amount of work per processor remains constant. Experiment configuration:

- Work per process: $n = 2^{19}, 2^{20}, 2^{21}$
- Processes: $p = 1, 2, 4, 8$

Expected behavior:

- Execution time remains approximately constant.
- Efficiency remains relatively stable.

3.2.1 Execution Time Results for Weak Scaling

Table 2 shows example results from weak scaling.
Figure 2 shows the weak scaling efficiency and speedup of different problem sizes under 1,2,4,8 processes.

4 Cost Analysis (Task 2.3)

The cost of the tree-structured reduction algorithm can be expressed as

$$T(p) = ?r + ?a$$

Where

- r = time for one receive operation

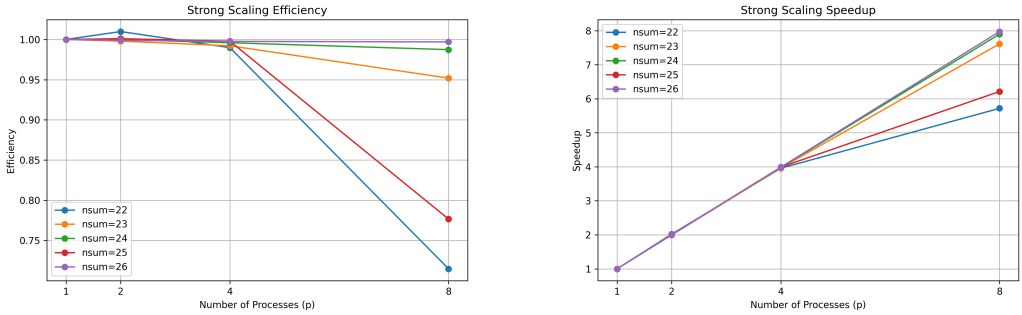

Figure 1: Strong Scaling Efficiency and Speedup

Table 2: Weak Scaling Execution Times

Processes (p)	Problem Size	Time (s)
1	2^{19}	0.0032
2	2^{19}	0.0017
4	2^{19}	0.0009
8	2^{19}	0.0004
1	2^{20}	0.0064
2	2^{20}	0.0033
4	2^{20}	0.0017
8	2^{20}	0.0009
1	2^{21}	0.0130
2	2^{21}	0.0065
4	2^{21}	0.0033
8	2^{21}	0.0017

- a = time for one addition

Process 0 performs operations equal to the number of tree levels. Since the algorithm has $\log_2(p)$ levels:

- Number of receives = $\log_2(p)$
- Number of additions = $\log_2(p)$

Therefore

$$T(p) = \log_2(p)r + \log_2(p)a$$

Example for $p = 8$

$$\log_2(8) = 3$$

Thus

$$T(p) = 3r + 3a$$

This matches the program output:

```
Process 0 performed 3 receives and 3 additions
Cost formula: T(p) = 3r + 3a
```

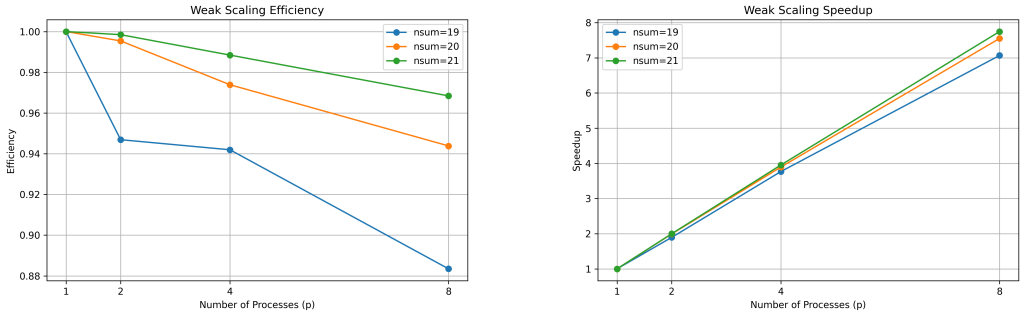

Figure 2: Weak Scaling Efficiency and Speedup

5 Speedup and Efficiency Analysis (Task 2.4)

Theoretical execution times are given as

Sequential time

$$T_s(n) = n^2$$

Parallel time

$$T_p(n, p) = \frac{n^2}{p} + \log_2(p)$$

Speedup

$$S = \frac{T_s}{T_p}$$

Efficiency

$$E = \frac{S}{p}$$

Tables were generated for

- $n = 10, 20, 40, 80, 160, 320$
- $p = 1, 2, 4, 8, 16, 32, 64, 128$

Below are the tables of Speedup table-3 and Efficiency table-4 for finding results based on the above-mentioned n and p values.

Table 3: Speedup Table

$p \backslash n$	10	20	40	80	160	320
1	1.00	1.00	1.00	1.00	1.00	1.00
2	1.67	1.82	1.90	1.95	1.98	1.99
4	2.22	2.86	3.33	3.64	3.81	3.90
8	2.35	3.64	5.00	6.15	6.96	7.44
16	2.16	3.81	6.15	8.89	11.43	13.33
32	1.88	3.56	6.40	10.67	16.00	21.33
64	1.62	3.17	6.04	11.03	18.82	29.09
128	1.41	2.79	5.47	10.49	19.39	33.68

Table 4: Efficiency Table

$p \backslash n$	10	20	40	80	160	320
1	1.00	1.00	1.00	1.00	1.00	1.00
2	0.83	0.91	0.95	0.98	0.99	0.99
4	0.56	0.71	0.83	0.91	0.95	0.98
8	0.29	0.45	0.62	0.77	0.87	0.93
16	0.14	0.24	0.38	0.56	0.71	0.83
32	0.06	0.11	0.20	0.33	0.50	0.67
64	0.03	0.05	0.09	0.17	0.29	0.45
128	0.01	0.02	0.04	0.08	0.15	0.26

5.1 Observations

For fixed n:

- Speedup initially increases with p
- Eventually saturates due to communication cost
- Efficiency decreases as processors increase

For fixed p:

- Speedup improves with larger problem sizes
- Efficiency increases because computation dominates communication

6 Scalability Analysis (Task 2.5)

The sequential and parallel execution times are:

$$T_s(n) = n, \quad T_p(n,p) = \frac{n}{p} + \log_2(p)$$

The efficiency is:

$$E = \frac{T_s(n)}{p \cdot T_p(n,p)}$$

To maintain constant efficiency while increasing the number of processors, the problem size must also increase. From the model, the required growth follows:

$$n \propto p \log_2(p)$$

If the number of processors increases by a factor $k > 1$, then:

$$p_{\text{new}} = k \cdot p$$

and the scaling factor for the problem size is:

$$\frac{n_{\text{new}}}{n} = \frac{kp \log_2(kp)}{p \log_2(p)}$$

Example

For $p = 8$ and $k = 2$:

$$p_{\text{new}} = 16$$

$$\frac{n_{\text{new}}}{n} = \frac{16 \cdot \log_2(16)}{8 \cdot \log_2(8)} = \frac{16 \cdot 4}{8 \cdot 3} = \frac{64}{24} \approx 2.67$$

For $n = 67108864$:

$$n_{\text{new}} \approx 2.67 \times 67108864 \approx 178956971$$

This shows that when the number of processors is doubled from 8 to 16, the problem size must increase by approximately a factor of 2.67 to maintain the same efficiency. This confirms that the problem size must scale as $p \log_2(p)$ due to communication overhead in the parallel reduction.

7 Discussion

The results show that the execution time decreases as the number of processors increases. However, the speedup is slightly lower than the ideal linear speedup.

This behavior is expected because parallel programs experience overhead from:

- Communication between processes
- Synchronization delays
- MPI message latency

In strong scaling experiments, efficiency decreases as more processors are added. This occurs because communication overhead becomes significant relative to computation.

In weak scaling experiments, the execution time remains approximately constant as the number of processors increases. This indicates good scalability, since each processor performs roughly the same amount of work.

Overall, the results match the theoretical expectations for parallel reduction algorithms.

8 Conclusion

This assignment implemented a parallel global summation program using MPI. The program distributes the computation across multiple processes and combines the results using a tree-structured reduction algorithm.

Key findings include:

- Block partitioning distributes workload evenly.
- Tree-based reduction reduces communication complexity to $O(\log p)$.
- Strong scaling shows improved performance but decreasing efficiency.
- Weak scaling demonstrates stable performance as processors increase.

The experiments demonstrate fundamental concepts in parallel computing, including scalability, speedup, and communication overhead.

9 References

- Pacheco, P. S. *An Introduction to Parallel Programming.* Amsterdam: Morgan Kaufmann, 2011.
- Grama, A., Gupta, A., Karypis, G., Kumar, V. *Introduction to Parallel Computing*
- MPI Forum Documentation: `https://www.mpi-forum.org`
- Lecture notes on Parallel and Distributed Programming